

Within a Pixel: Building Virtualized Geometry You Can Check

What I learned building a Nanite-style renderer from scratch, and trying to prove (not just hope) that every cluster it draws is within a pixel of the original.

Abir Deol · University of Alberta, Computing Science · October 2026 · github.com/apollo-2006/colossus

WORKING DRAFT: BEING REVISED, AND IT WILL CHANGE

Abstract. Colossus is a virtualized geometry renderer I wrote from scratch in C++20 and Vulkan, with a WebGPU port that runs in the browser. It follows the design Epic described for Unreal Engine 5's Nanite: a crack-free hierarchy of triangle clusters, streamed from disk, where the GPU picks the coarsest detail for each cluster that still stays within a pixel of the original. Most of that is a reimplementation, and I want to credit it as one. What this write-up is really about are the places where I tried to go a step further, and the places where that turned out to be harder than I expected. The big one is the error bound: instead of an estimate, every level carries a proven upper bound on how far it is from the full-detail surface, and I check that claim in rendered images instead of just asserting it. At a one pixel threshold, every pixel whose coverage changes touches the full-detail silhouette. The rest is what happened when the renderer met real content: a textured museum scan, an animated character, and a photoscanned forest made of millions of separate needles. Each one broke an assumption I didn't know I'd made, and each fix taught me something about where proofs end and measurements have to take over. I also measured it against meshoptimizer's reference builder through the same renderer: colossus keeps silhouettes tighter at the same triangle count, and meshoptimizer shades better. 900 statues, 15.9 billion triangles in total, draw at 1920x1080 in 1.51 ms on an RX 9070 XT.

1. Why build it at all

I started colossus because I wanted to really understand Nanite, and the only way I trust myself to understand something is to build it and watch it break. Brian Karis's 2021 talk [1] lays out the idea really clearly: split a mesh into small clusters, simplify groups of them into coarser clusters, repeat until there's one cluster left, and let the GPU walk the result every frame, deciding for each cluster whether it (or something coarser) is good enough for the pixels it covers. When I watched it, it was easy to nod along. It was a lot harder to say exactly why the cut it picks never has cracks, or what "good enough" is actually promising.

Honestly, this started as something I didn't even plan to put on GitHub. It was just for learning, and then I got a little obsessed. So colossus ended up being two things at once. It's a working renderer: clustering, simplification, graph partitioning, streaming from disk, culling, two rasterizers, virtual shadow maps, ambient occlusion and bounce light, and temporal antialiasing, all in the repository with no engine underneath. And it's a long exercise in replacing "it looks right" with something I can actually check. This write-up walks through the pieces in the order they taught me something, and I've tried to be upfront about which claims are proven, which are measured, and which are still hopes.



Figure 1. 900 instances of Lucy (28 million triangles) and the XYZ RGB dragon (7.2 million): 15.9 billion triangles at full detail, drawn in 1.51 ms with soft shadows, ambient occlusion, bounce light and antialiasing. About 4.8 million triangles reach the screen.

2. The hierarchy

The builder reads a scan and turns it into a directed acyclic graph of clusters. Each cluster holds at most 128 triangles and 128 vertices, cut out by recursively bisecting the triangle graph. I started with a greedy grower instead, and it kept leaving about 8% of clusters as small walled-in pockets, which wastes GPU work since every cluster is a 128-triangle job. Bisection fills nearly all of them: the dragon's 56,404 leaf clusters average 128.0 triangles. Each level then groups about eight neighbouring clusters, simplifies each group to half its triangles, and splits the result back into clusters.

The trick that lets any mix of levels meet without cracks is locking. A vertex used by two groups isn't allowed to move, so a group's outline is identical before and after it's simplified, and a coarse group always meets its neighbours, coarse or fine, edge for edge. Groups fall differently at every level, so no outline sticks around for long. I don't just trust this: `--check` builds every cut at 25 thresholds and looks for any edge used by a single triangle whose ends aren't both on one of the scan's own holes. Every model in this write-up passes at every cut.

Picking a level then needs no tree walk at all. Each cluster stores its own error and its parent group's, and as long as both only grow toward the root, the rule

```
own error on screen <= threshold < parent error on screen
```

picks exactly one level along every path from a leaf to the root. One GPU thread per cluster checks it on its own.

3. An error bound you can prove

The interesting word in that rule is "error". Simplifiers usually report quadric error [2], which is a mean squared distance, and it routinely understates the worst spot by a factor of two or more. If that's the number deciding when

a cluster is good enough, then "within a pixel" is really just a hope.

So colossus bounds the two-sided Hausdorff distance between each group and its simplification from above. The key fact that makes this work: over a single triangle, the distance to any one triangle of the other mesh is a convex function, so it's largest at the triangle's corners. That means the distance to the whole mesh is at most the smallest such corner bound over nearby triangles. Any piece whose bound could still beat the worst distance found so far gets split in four and refined, until every bound is within 10% of it. The result is always an upper bound, and refining only ever tightens it. A test checks it against brute force on about 94,000 random points. Errors add up through the levels (a coarse level's error is its children's plus its own), so they only grow, and the grid snapping used to store positions adds half a grid step's diagonal, so that what's actually drawn, not just what was built, stays inside the bound.

The second half of the claim is the projection onto the screen. The usual estimate, error divided by distance, treats every point as if it sat right on the optical axis. But a point at position p moved by e moves up to $\text{lod_scale} \cdot e \cdot |p| / z^2$ pixels, which grows toward the edges of the screen, by up to 2.2 times in a 16:9 corner. Colossus judges each sphere by its nearest depth and by the angle of its point farthest off axis, capped at the corner. Once I made the projection honest, it turned out my old "one pixel" had really been up to two.

Checking it in images

A proof about distances isn't a statement about pixels yet, so I measured it. A script renders five views twice each: once at a threshold of 0.05 pixels, which is full detail for all practical purposes, and once at the threshold being tested, with every source of frame-to-frame noise switched off so that two renders of the same cut are byte-identical. Then it compares them two ways. A coverage view draws white wherever a model is, and for every pixel whose coverage differs, I measure its distance from the full-detail silhouette. Separately, NVIDIA's FLIP [3] compares the shaded images the way a person would see them.

view	full detail	at 1 px	coverage differs	farthest from silhouette	FLIP	pixels off by > 8/255
Lucy, close	6.21M	374.1k	0.011%	1.41 px	0.0119	1.06%
dragon, whole	3.72M	273.9k	0.007%	0.00 px	0.0044	0.68%
Washington, face	2.18M	313.5k	0.010%	0.00 px	0.0088	0.59%
fox, walking	554.0k	62.3k	0.001%	0.00 px	0.0003	0.03%
crowd of nine	21.66M	1.10M	0.020%	1.41 px	0.0096	1.77%

At one pixel, in every view, every pixel whose coverage changes touches the full-detail silhouette, at most a diagonal step away. The silhouette only strays further from 4 pixels in the crowd, and 8 to 16 pixels elsewhere. The differences that are left come from shading, not geometry (Figure 2): a coarse triangle blends its corners' normals over a bigger area, and the bound doesn't say anything about normals. I think that's the most honest summary of the bound: it's real, it holds, and it's conservative. Lucy draws 374,000 triangles at one pixel, and at sixteen pixels, with only 39,000, her outline is still within 2.3 pixels of full detail. A guarantee costs triangles, and now I know roughly how many.

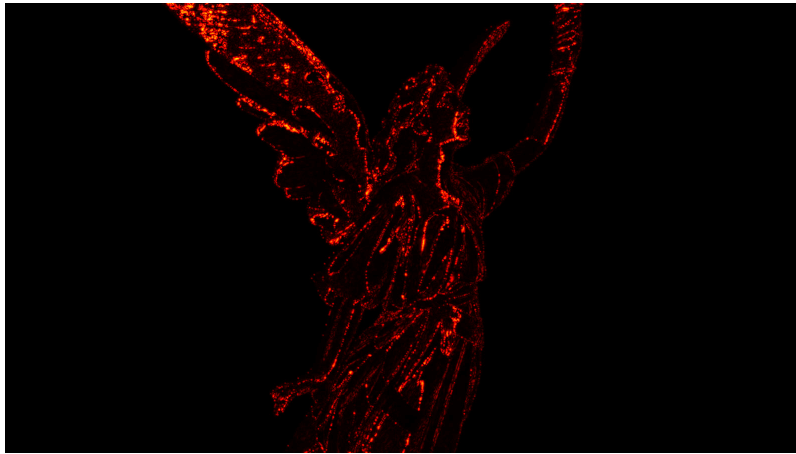


Figure 2. Where Lucy at one pixel differs from full detail, by FLIP: along creases and fine folds, where coarser triangles interpolate normals over larger spans. Silhouettes and broad surfaces are clean.

4. Drawing and streaming

The renderer follows the published design pretty closely, so I'll keep this part short. Instances are culled in cells of 64, then one by one, and clusters are culled for level of detail, the frustum, a normal cone, and occlusion against last frame's depth pyramid, then again against a fresh one for anything that just became visible. Clusters bigger than 32 pixels go to mesh shaders and smaller ones to a compute rasterizer, since pixel-sized triangles are exactly what fixed-function hardware handles worst. Both write depth and a triangle id into one 64-bit visibility buffer with an atomic max. Shading looks each pixel's triangle back up, intersects the pixel's ray with it for exact barycentrics, and lights it with virtual shadow maps (a 14-level clipmap of 128x128 texel pages), screen-space ambient occlusion and a single bounce of indirect light.

Only bounds and errors stay in memory, 48 bytes a cluster. The geometry itself lives in pages, one per group, that stream into a fixed pool when they're needed. Cuts stay whole while streaming because a page is only resident while the pages of its coarser stand-ins are, so whatever has loaded, exactly one cluster draws on every path. Pages are bit-packed per cluster: positions as offsets on a grid using only the bits the cluster's size needs, normals as 11 plus 11 bit octahedral, and triangle indices in the bits the vertex count needs. I tried to squeeze them further, but general-purpose compressors find almost nothing left (zstd at level 19 saves 2%), and the obvious next step, storing triangles as a base index plus small offsets, saves under 5% of the triangle data (under 2% of a page), because one wide triangle sets the width for its whole cluster. Real gains would need sequential strip encodings decoded into shared memory, which would give up the random access the shading pass depends on. That's where I've left this thread for now.

The image check from section 3 really earned its keep here, on a bug no proof could ever have found. In the Washington close-up, one pixel deep inside the statue showed the sky. It went away with either rasterizer on its own, so it had to be sitting on a seam between a cluster drawn by the hardware and one drawn in compute. Both snap vertices to $1/256$ of a pixel, but they don't compute positions bit for bit the same way, so every now and then a shared vertex lands one step apart and leaves a sliver too thin to see, except where it crosses a pixel centre. Making the two agree exactly isn't possible from a shader, so instead each compute-drawn triangle grows by one step and the depth test settles the overlap. Over sixteen crowd views, the holes compared to hardware-only rendering went from nine to none, at the price of 913 silhouette pixels whose centres sit within $1.5/256$ of an edge.

5. Textures on a surface that keeps changing

The Stanford scans I started with have no texture coordinates, so their marble and bronze are procedural. To be honest about real content I needed a textured scan, and the Smithsonian American Art Museum has released Horatio Greenough's 1840 statue of George Washington under CC0: 17 million triangles and a 4096x4096 texture.

Simplifying a textured mesh means a vertex isn't one thing anymore. On a seam, the same position has different texture coordinates in each chart, so colossus keeps them per wedge: a vertex as one chart sees it. A collapse moves each corner onto the kept vertex's wedge in the same chart, and it's refused if there isn't one, so seams only slide along themselves and every level's coordinates are exactly the original's. That rule worked perfectly on my test sphere with eight charts, and then stalled completely on the real scan, at 28 thousand triangles with every group stuck. A photogrammetry atlas has hundreds of tiny charts, and the rule pinned all of them. The fix was to let a stuck group's seams move, keeping each corner's old coordinates on a new wedge at the kept vertex. Positions are still shared, so there are still no cracks, and the texture slips across a seam by no more than the collapse moved, which the error already counts.

Textures stream the same way geometry does. Each mip level is cut into 128x128 tiles (120 texels plus a four texel border, so bilinear filtering never leaves a tile), compressed to BC1, and every tile is a page in the same streamer, depending on the coarser tile above it. The rule that keeps cuts whole now also guarantees that a coarser copy is always loaded, so a missing tile just draws its parent until it arrives. Shading decodes the BC1 blocks by hand straight from the page pool, which sounds expensive but isn't: textured shading costs 0.61 ms on the scan against 0.78 ms for the procedural marble it replaces.

Poly Haven's models also come with normal and roughness maps. Those are one channel of data each (a normal map's x and y, and a roughness), stored as BC4, whose blocks are the same size as BC1's, so every tile stays the same size and the streamer never even noticed. The tangent frame is built per pixel from the triangle's own corners and texture coordinates, so nothing extra has to be stored. Khronos's NormalTangentMirrorTest model, which exists to catch exactly the kind of convention mistakes this invites, renders correctly, mirrored coordinates included.



Figure 3. Horatio Greenough's George Washington (1840), the Smithsonian American Art Museum's scan, in its own texture, streamed in tiles beside the geometry. 0.80 ms at 1920x1080.

6. Skinning, and where proof stopped working

An animated character was the first place the provable approach failed outright, and it's the part of this project I learned the most from. Linear blend skinning moves each vertex by a weighted blend of joint transforms. A coarse triangle's interior blends its corners' weights linearly, while the fine surface it stands in for blends its own, and the two disagree wherever the weights change quickly. Bounding that disagreement means assuming the worst mix of weights against the worst separation of joints. I worked out that bound, implemented it carefully, and measured it: 900 animated foxes drew 51.9 million triangles in 8.86 ms, against 1.33 million for the same foxes standing still. At coarse levels the worst case is a whole body length, so no fox could ever go coarse, even when it only covered thirty pixels.

The way out was realizing that the animations are already known when the model is built. So the builder measures each group's simplification in pose: the fine and the simplified surface, skinned by the same joints, at sixteen points in each animation and in the rest pose, with the worst distance added to the group's error. That alone wasn't enough, because the simplifier itself was blind to motion. Its quadrics come from the rest pose, where the skin around a knee is flat, so it happily removed exactly the vertices a bent knee needs. Summing quadrics over several poses, which Mohr and Gleicher proposed for articulated meshes [4], fixed it. The median error at the first level dropped from four times the rigid model's to the same as it, and a crowd of 900 foxes now draws in 1.73 ms.

This error is measured over the animations the model ships with, not proven. I want to be really clear about that, because the rest of the system is careful about the difference. A pose the builder never saw could exceed it.

The rest of skinning was engineering. Every animation is posed at sixteen phases on the CPU once a frame, so a million foxes share a few dozen poses. Culling spheres follow the joints: the level-of-detail test takes the smaller of two bounds that both nest from leaf to root (one follows the most weighted joint, one grows the rest pose by how far the joints move), and the smaller of two nested bounds still nests. Antialiasing finds where a pixel was last frame through its triangle posed as it was last frame, since skinning has no inverse.

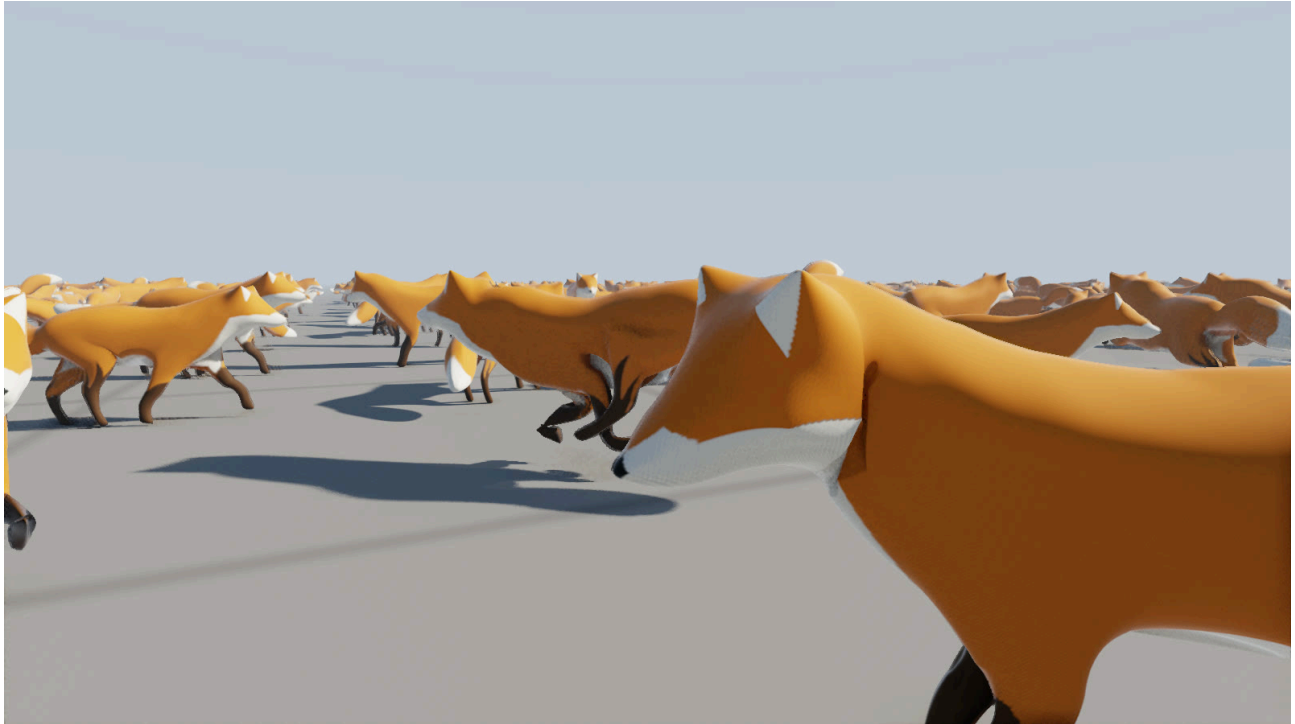


Figure 4. 900 Khronos glTF foxes, subdivided to 590 thousand triangles each, walking, running and looking around, each cluster still choosing its own level.

7. Foliage, and a bug that was really a definition

Statues are the friendliest content there is: one closed surface, every triangle sharing its edges with three others. For a real scene I wanted the opposite, so I picked a forest.

A photoscanned pine from Poly Haven's CC0 pine forest collection is about seven million triangles, and most of them are needles: thousands of separate little pieces. Edge collapse is designed never to delete an island, so simplification just stalled with every needle intact. At the tenth level, every one of the pine's 13 thousand groups was stuck. Two changes got it moving. The partitioner, which used to group clusters only by shared edges, now also links each island to its nearest clusters, weighted below any real connection, so needles form groups with their neighbours. And when collapses stall, vertex clustering takes over, snapping each grid cell's vertices together while every vertex on the group's outline stays exactly where it is, so its neighbours still meet it. The needles merge into coarser shapes.

Then the crack check found two cracked edges, at a few of its 25 cuts, in a 24.5 million triangle hierarchy. I want to tell this one properly, because I think it's the most useful thing in here. My first two theories were wrong. What finally worked was instrumenting the builder: after every group's simplification, report any rim edge in the output that touches a vertex that wasn't on a rim in the input. It pointed not at the new vertex clustering, but at the old edge collapse code, in groups whose input had come from clustering. Vertex clustering keeps duplicate triangles on purpose, so every edge's use count keeps its parity, and that means a rim edge can be used three times. The

simplifier, written months earlier for clean scans, defined a rim edge as one used exactly once, so it read those edges as interior and let their vertices wander inward. The definition was right for the meshes I had when I wrote it, and wrong for the meshes I had now. Changing "used once" to "used an odd number of times" in two places made every cut of the pine crack-free.

A crown that thins

Merged needles cover less area, so from a distance a crown thins out. That's the well known weakness of simplifying foliage, and it took me a while to realize that my error bound actually allows it. A Hausdorff distance asks how far each surface is from the other. Take away half a crown's needles and every missing needle is still close to one that's left, so the bound is perfectly happy while the tree goes see-through. In the coverage view, one pine 200 metres away kept 88% of its full-detail coverage at one pixel, 72% at four and 45% at sixteen.

My first fix grew back what clustering left behind: each surviving piece takes on the area of the needles that merged into it and scales up around its centre to match, with a new vertex for each one it moves, and the outline's vertices left alone so cuts stay whole. It helped a little (45% became 50% at sixteen pixels), but most of the thinning happens earlier, in ordinary edge collapses that turn needles into slivers. Growing after those too kept compounding level after level until groups got stuck and the pine never reached a root, so I took that part back out.

The fix that actually worked changes what the error means for foliage. A group made mostly of open rims, which is what needle cards look like and closed scans don't, now also counts the area it lost: its error is at least the square root of that area. A cut drawn at t pixels then loses at most about t^2 pixels of coverage per group, and the renderer refines wherever it would lose more. Counting all of the lost area, the same pine kept 95% at one pixel, 85% at four and 60% at sixteen, but for two and a half times the triangles. A lot of a crown's lost area is hidden behind other needles anyway, so the default counts half of it (a cut at t pixels then loses about $(2t)^2$ pixels per group): 91% at one pixel, 74% at four and 51% at sixteen, for 1.2 to 1.4 times the triangles. It costs triangles, honestly: the forest draws 113 million where it drew 90. The statues, being closed surfaces, build exactly as they did before, byte for byte.

With all of that in, all 55 models in the collection build crack-free at every cut. Scattered over 100 by 100 metres, the forest is 14,780 instances and 753 million triangles at full detail, and it draws in 7.5 ms at 1920x1080 with Poly Haven's normal and roughness maps (Figure 5). That's about five times the statues' frame, and I think it's the honest number to show: crowns are mostly pixel-sized triangles that hide very little behind them, and keeping them full means drawing them. Its rasterizer turned out to be limited not by triangles, but by the visibility buffer: with the writes taken out, it gets through 364 million triangles in 1.33 ms, and a read or an atomic per covered pixel costs about the same. Resolving each cluster in shared memory first, the obvious fix, was slower at every tile size I tried, because the 64-bit shared atomics and the occupancy they cost outweighed the traffic they saved. That one's still open.

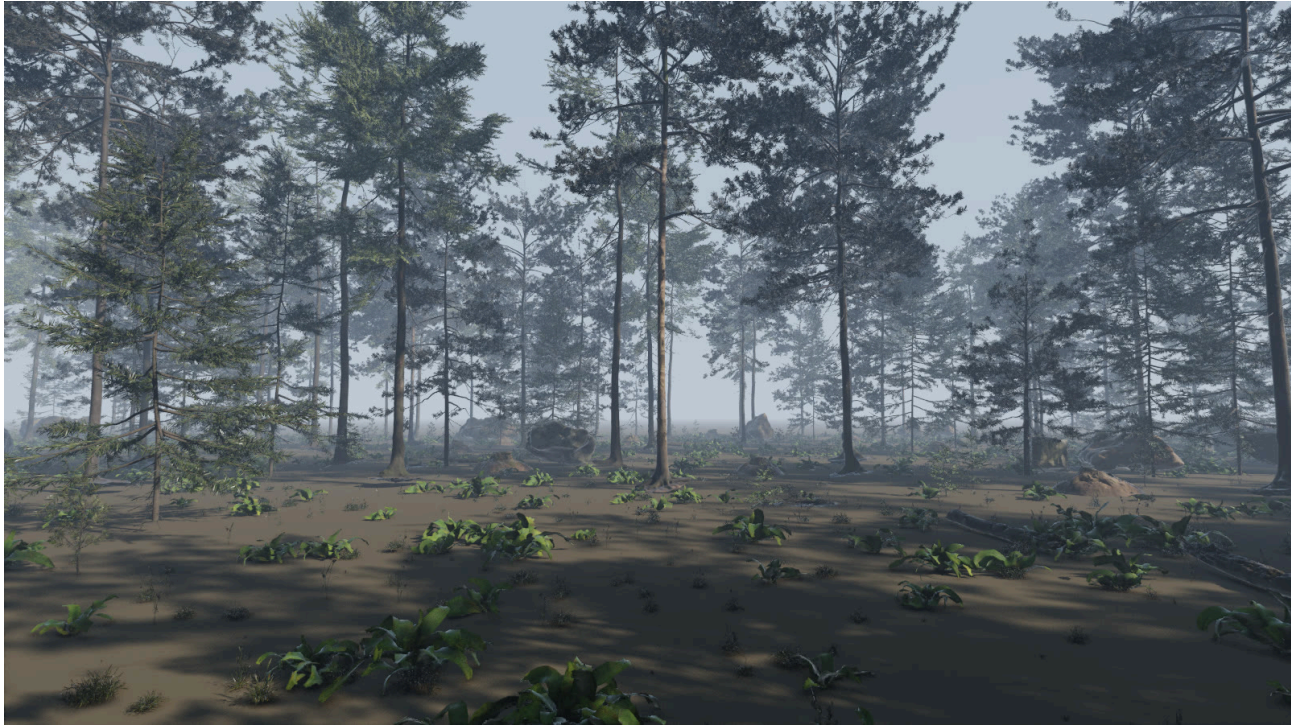


Figure 5. Poly Haven's CC0 pine forest collection, 55 models scattered into 14,780 instances: 753 million triangles at full detail, 7.5 ms at 1920x1080.

8. In the browser

The WebGPU port runs the same pipeline without mesh shaders, 64-bit atomics or ray queries: big clusters go through a plain render pipeline, the compute rasterizer runs twice (depth first with a 32-bit atomic max, then the triangle wherever its depth won), and pages stream over HTTP range requests. The tightest squeeze was the limit of 16 storage buffers per shader stage, which one layout already hit, so skinning's joints ride along in the page table buffer instead of getting a buffer of their own. Skinning's unused branches still cost the statues 0.1 ms in registers, so every geometry pipeline also exists compiled without it, and the renderer uses that set whenever nothing on screen is skinned. Phones without WebGPU get a WebGL2 fallback that runs the same level-of-detail test on the CPU. In Chrome, 900 statues take 1.73 ms of GPU time once streaming settles.

9. Measured against meshoptimizer

A number like 1.51 ms doesn't mean much without something to compare it to, so I built Lucy and the dragon a second way, with meshoptimizer's clusterlod, the reference Nanite-style builder that a lot of graphics programmers know, at its own default settings. A small tool converts its hierarchy into colossus's format, which works because the two agree on what a group is and when a cluster draws. Both builds then go through the same renderer, cameras and measurements, so the only thing that differs is the builder. Both are crack-free at every cut.

view	builder	threshold	triangles	coverage differs	farthest from silhouette	FLIP
Lucy, close	colossus	1 px	374.1k	0.011%	1.41 px	0.0119
Lucy, close	clusterlod	0.5 px	365.0k	0.012%	1.41 px	0.0097
dragon, whole	colossus	1 px	273.9k	0.007%	0.00 px	0.0044
dragon, whole	clusterlod	0.5 px	240.9k	0.012%	0.00 px	0.0038
crowd of nine	colossus	4 px	235.5k	0.076%	2.24 px	0.0178
crowd of nine	clusterlod	2 px	230.9k	0.132%	8.00 px	0.0167

The result wasn't the one I was hoping for, and I think it's more useful because of that. At about the same number of triangles, meshoptimizer shades as well or better, while colossus keeps silhouettes tighter, with half the coverage change or less and never more than 2.24 pixels out where clusterlod's strayed 8 to 9 in the crowd. Its threshold is an estimate, so its "one pixel" draws about what my two to four do. Through the same renderer a triangle costs the same, so at one pixel the crowd takes 1.54 ms against 1.42: that difference is basically the price of the guarantee. meshoptimizer also builds 1.6 to 3.4 times faster, so that one's all theirs.

The comparison also handed me something to try. Pricing each collapse by how much it changes the normal as well as the position lowers FLIP by 5 to 10% at equal triangles, without touching the proof, since the bound is still measured afterwards. It also makes each pixel of threshold cost more triangles, so it's an option instead of the default. Then I tried the textbook version, Hoppe's attribute quadrics, and at equal triangles it was no better at all. That told me where the gap actually comes from: not the order of collapses within a group, but which level each part of the model draws at. Colossus chooses that by geometric distance, so it spends its triangles on silhouettes, while meshoptimizer folds shading into its error and spends them where the normals change. Closing the gap would mean making shading part of the error, which is a different promise, not a better one.

10. What it costs, and what it doesn't do

A demo always shows the best case, so here are the worst ones I know about, measured where I could.

- Every number here comes from one GPU, my RX 9070 XT on Mesa's RADV driver. I haven't measured other vendors yet; NVIDIA and Intel are next.
- The bound covers geometry, not shading. A normal or a shadow can shift by more than a pixel's worth where it depends on detail a cut removed.
- Proving bounds makes building slow. Lucy takes 4.4 minutes, down from 5.7 once the bound's nearest-triangle searches learned to skip triangles that can't be nearer, with byte-identical output.
- Skinned errors are measured over the shipped animations, not proven. Animated instances are expensive in the shadow maps, which redraw their pages every frame: a million foxes take 7.29 ms, 4.10 of it shadows.
- Foliage is the most expensive content here, and its coverage is measured as triangle area, which overlapping needles overstate.
- A scene whose pages outgrow the pool thrashes: coarse stand-ins show while pages wait, and shadow pages redraw every frame. Scene files can ask for a bigger pool.
- Textures have no alpha yet, so a card meant to be cut out draws as its whole quad.

11. What I'd tell myself at the start

Most of the hard problems in this project weren't where I expected them. The cluster hierarchy, which looks like the clever part, worked within days. What took weeks was every place where a definition I'd written for one kind of

mesh quietly stopped being true for the next: one texture chart per seam, one pose per surface, one triangle per rim edge, a distance as the whole of what an error means. The habit that helped the most was refusing to accept "it looks right": a crack check that runs on every build, a measurement that renders the claim instead of just restating it, and instrumenting the builder whenever a theory failed twice. I didn't expect a renderer to teach me that, but it did. And if you've read this far, thank you, genuinely.

Acknowledgements and credits

The design follows Brian Karis, Rune Stubbe and Graham Wihlidal's Nanite [1]. Lucy and the dragon are from the Stanford 3D Scanning Repository (and XYZ RGB Inc. for the dragon). George Washington is the Smithsonian American Art Museum's scan, released CC0 through Smithsonian Open Access. The fox is from the Khronos glTF sample assets: model by PixelMannen (CC0), rigging and animation by tomkranis, glTF conversion by AsoboStudio and scurest (CC BY 4.0). The forest is Poly Haven's pine forest collection (CC0). Image comparisons use NVIDIA's FLIP [3].

References

- [1] B. Karis, R. Stubbe, G. Wihlidal. Nanite: A Deep Dive. SIGGRAPH Advances in Real-Time Rendering in Games, 2021.
- [2] M. Garland, P. Heckbert. Surface Simplification Using Quadric Error Metrics. SIGGRAPH 1997.
- [3] P. Andersson, J. Nilsson, T. Akenine-Möller, M. Oskarsson, K. Åström, M. Fairchild. FLIP: A Difference Evaluator for Alternating Images. Proceedings of the ACM on Computer Graphics and Interactive Techniques, 2020.
- [4] A. Mohr, M. Gleicher. Deformation Sensitive Decimation. Technical report, University of Wisconsin, 2003.